

Gymnázium Praha 6, Arabská 14

Programování



Ročníkový projekt 3.E

Předmět: Programování

Téma: Cell Expansion Wars

Autor: Šimon Petříček, Reuel Dejmal, Jakub Hruška

Třída: 3.E

Školní rok: 2025/26

Vedoucí práce: Mgr. Jan Lána

Třídní učitel: Mgr. Jan Tuzar

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

Anotace

Tento projekt představuje webovou hru s názvem Cell Expansion Wars. Hra nabízí tři herní úrovně a multiplayer režim pro více hráčů. Součástí aplikace je také systém přihlašování, který zajišťuje bezpečnost a ukládání herních výsledků.

Promyšlená herní mechanika zajišťuje dlouhodobé udržení pozornosti hráče. Celkový koncept vychází z osvědčených herních systémů a moderního přístupu k návrhu her.

Anotace

This project presents a web-based game called Cell Expansion Wars. The game offers three levels for players to explore, as well as a multiplayer mode supporting multiple players simultaneously. The application also includes a login system to ensure security and track player results.

Obsah

Úvod	5
1 Popis hry	6
1.1 Herní myšlenka	6
1.2 Typy buněk	6
1.3 Herní mechaniky	7
1.4 Podmínky výhry a prohry	7
1.5 Úrovně a obtížnosti	7
2 Technická část	8
2.1 Použité technologie	8
2.2 Struktura souborů	9
2.3 Frontend: popis herní logiky	9
2.4 Backend: popis serverové části	12
2.5 Komunikace frontendu a backendu	13
3 Grafika a design	14
3.1 Grafický koncept buněk	14
3.2 Technické zpracování textur	14
3.3 Animace pomocí spritesheetů	15
3.4 Grafika vojáků	15
3.5 Herní prostředí a pozadí	15
4 Využití umělé inteligence při vývoji	16
4.1 Detekce bodu u úsečky	16
4.2 Bezpečné načítání obrázků pomocí Promise	17
4.3 Pravidelné odesílání vojáků pomocí setInterval	17
5 Srovnání s podobnými hrami	18
6 Závěr	19
6.1 Co jsme se naučili	19
6.2 Možnosti dalšího rozvoje	19
6.3 Zhodnocení projektu	19
Zdroje	20

Úvod

Hraní her patří mezi nejrozšířenější způsoby trávení volného času, a to jak v online prostoru, tak i v reálném světě. V posledních letech byl zaznamenán výrazný nárůst popularity her, které slouží jako prostředek odreagování po pracovním nebo školním dni. Pod pojmem odreagovací hra si lze představit hru, při níž hráč dokáže uvolnit mysl a načerpat novou energii. Na druhé straně, a zdánlivě protikladně, klademe důraz na strategické myšlení a aktivní zapojení kognitivních schopností. Kombinování těchto dvou podmínek splňuje jen málokterá hra, a to právě kvůli jejich protichůdným přístupům.

Výsledkem naší práce je hra, která tyto dvě podmínky propojuje. Pracovali jsme na ní jako na ročníkovém projektu a v průběhu jeho prezentace si představíme metody a části kódu použité při jejím vývoji. Popis hry je záměrně jednoduchý: hráč ovládá jednu až tři buňky a postupně z nich vysílá vojáky, aby dobýval okolní buňky. Grafické zpracování hry umožňuje hráči odpočinout si, avšak pro dosažení vítězství je nezbytné zapojit strategické myšlení.

Z technického hlediska je projekt rozdělen na frontend a backend. JavaScript zajišťuje herní logiku: vojáci se pohybují plynule po obrazovce a po dosažení cílové buňky ji buď posilují, nebo oslabují. Při zničení nepřátelské buňky se tato přebarví na barvu útočníka. Hra obsahuje tři typy buněk: zelené ovládané hráčem, fialové ovládané počítačem a modré neutrální. Každý herní level nabízí odlišné rozmístění a počet buněk. Backend je postaven na frameworku Django s asynchronními pohledy využívajícími klíčová slova `async` a `await`, které obstarávají registraci, přihlášení, správu profilů a vytváření multiplayerových her.

Hra nabízí tři herní levely se stupňující se obtížností a multiplayerový režim pro hraní s více hráči v reálném čase. Tento projekt byl náročnou výzvou, která posunula naše znalosti programování na novou úroveň.

Vývoj počítačových her představuje komplexní disciplínu propojující programování, grafický design, matematiku i uživatelskou zkušenost. Úspěšná hra musí být nejen technicky funkční, ale také dostatečně zábavná a intuitivní pro cílovou skupinu hráčů. Při návrhu projektu proto bylo důležité hledat rovnováhu mezi jednoduchostí ovládání a dostatečnou herní hloubkou.

1 Popis hry

1.1 Herní myšlenka

Brain Cell Wars je strategická prohlížečová hra, v níž hráč ovládá určený počet mozkových buněk a snaží se dobýt celou hrací plochu. Dobývání ostatních buněk probíhá vysláním vlastních vojáků. Počet životů buňky přímo koreluje s počtem vojáků, kteří se v ní aktuálně nacházejí.

Ve hře si hráč může procvičit taktické rozhodování a zkoušet nové strategie. Jednoduchost ovládání je klíčová pro dlouhodobé udržení zájmu hráče. Při hraní musí hráč neustále zvažovat možnosti útoku i obrany a reagovat na aktuální rozložení sil na herní ploše.

Důležitým cílem bylo vytvořit herní systém, který bude snadno pochopitelný i pro nové hráče. Zároveň však musí poskytovat dostatek možností pro plánování a vytváření různých strategií. Každé rozhodnutí hráče může ovlivnit průběh celé partie a vést k odlišnému výsledku.

1.2 Typy buněk

Hra rozlišuje tři typy buněk, od sebe vizuálně oddělených barvou a texturou. Hráč může ovládat výhradně buňky zelené barvy a pouze z nich odesílat vojáky na cílové buňky.

Modrá buňka reprezentuje neutrální entitu. Nepatří žádnému z hráčů a na začátku hry dostává určitý počet vojáků, kteří brání její dobytí. Modrá buňka nabízí strategickou příležitost k rozšíření území v průběhu hry.

Fialová buňka zastupuje počítačového nepřítele. Nepřítel odesílá jednoho vojáka přibližně každou sekundu. Výběr cílové buňky je řízen umělou inteligencí a probíhá náhodně. Fialová buňka může útočit jak na buňku hráče, tak na buňky neutrální, přičemž barva cílové buňky nehraje roli.

Rozdělení buněk do několika kategorií pomáhá hráči rychle se orientovat v herní situaci. Barevné odlišení zároveň umožňuje okamžitě rozpoznat vlastní i nepřátelské jednotky. Díky tomu je hra přehledná i ve chvílích, kdy se na obrazovce nachází větší množství objektů.

1.3 Herní mechaniky

Jednoduchost ovládání je klíčovým parametrem hry. Hráči stačí kliknout na vlastní buňku, čímž ji vybere, a poté kliknout na cílovou buňku, kam chce útok zamířit. Mezi oběma buňkami vznikne trvalá linka, po níž jsou vojáci odesíláni automaticky v pravidelných intervalech. Kliknutím na již existující linku ji hráč zruší. Díky tomuto mechanismu není nutné vysílat vojáky manuálně; hráč pouze nastaví správné linky a sleduje dění na hrací ploše.

V okamžiku, kdy hráčovi vojáci dorazí k nepřátelské nebo neutrální buňce, začínají snižovat počet jejích vojáků. Pokud počet vojáků klesne na nulu, buňka je dobytá a změní svoji barvu na barvu útočníka. Počítač funguje na obdobném principu, avšak nevytváří trvalé linky, nýbrž vysílá jednotlivé vojáky náhodným směrem přímo z každé své buňky. Při dobytí buňky obdrží nový vlastník pět vojáků jako výchozí stav. Navržené mechaniky podporují plynulost hraní a minimalizují počet zbytečných úkonů. Hráč se může soustředit především na strategická rozhodnutí namísto opakovaného klikání. Tento přístup zvyšuje dynamiku hry a zároveň zlepšuje celkový herní zážitek.

1.4 Podmínky výhry a prohry

Podmínky výhry a prohry jsou společné pro všechny herní úrovně. Hráč musí dobýt všechny cizí buňky, tedy jak modré neutrální, tak fialové nepřátelské. Po úspěšném dokončení úrovně se barva příslušného tlačítka v menu změní ze zelené na zlatou a v sekci úspěchů se zobrazí zlatý řádek dané úrovně. Po dokončení všech tří úrovní se hráči zobrazí výzva k vyzkoušení multiplayerového režimu. Multiplayer je přístupný od samého začátku hry, avšak hráč, který absolvoval všechny tři úrovně, přistupuje k němu s výrazně hlubší znalostí herních mechanik.

Podmínka prohry je stejně jednoznačná. Pokud nepřátelské fialové buňky dobudou všechny hráčovy zelené buňky, hra končí prohrou. Hráči se zobrazí oznámení s možností návratu do hlavního menu, odkud může hru zkusit znovu. Hra nezaznamenává počet pokusů a každý nový pokus začíná za stejných podmínek.

Jednoznačně definované podmínky výhry a prohry přispívají k lepší srozumitelnosti herního systému. Hráč má po celou dobu jasnou představu o svém aktuálním cíli. Díky tomu může efektivněji plánovat své další kroky a přizpůsobovat strategii aktuálnímu vývoji hry.

1.5 Úrovně a obtížnosti

Hra nabízí tři singleplayerové úrovně, přičemž každou z nich hraje v daný moment vždy jeden hráč. Jednotlivé úrovně se liší rozmístěním buněk na hrací ploše, počtem nepřátelských buněk i jejich výchozím počtem vojáků. Obtížnost postupně roste skrze několik herních parametrů: zvyšuje se počet nepřátelských buněk a jejich výchozí zásoby vojáků, případně klesá počáteční síla hráčovy buňky. Obtížnost nelze nastavit manuálně; hráč se s rostoucí náročností musí vypořádat výhradně zdokonalováním vlastních taktik a herních strategií.

Po úspěšném dokončení úrovně dojde k několika vizuálním změnám. Tlačítko dané úrovně v hlavním menu změní barvu ze zelené na zlatou a zobrazí se na něm ikona poháru. Hráče zároveň odmění animace zlatých konfet padajících po herní ploše. Splnění úrovně odemkne přístup k úrovni následující. Postup hráče je průběžně ukládán, takže dosažené výsledky zůstávají zachovány i po zavření hry.

Postupné zvyšování obtížnosti umožňuje hráči osvojit si základní herní principy ještě před setkáním s náročnějšími výzvami. Každá další úroveň klade vyšší nároky na rychlost rozhodování i správné rozložení jednotek. Tento princip motivuje hráče k neustálému zlepšování vlastních schopností.

2 Technická část

2.1 Použité technologie

Projekt se skládá ze dvou oddělených vrstev: frontendu a backendu. Každá vrstva slouží jinému účelu a obě dohromady tvoří funkční celek.

Frontend je vytvořen třemi základními webovými technologiemi. Jazyk HTML tvoří strukturální kostru celé stránky a určuje rozmístění všech viditelných prvků, jako jsou tlačítka pro přihlášení, herní plátno nebo obrazovky zobrazované po výhře a prohře. Vizuální stránku hry zajišťuje CSS, které definuje barvy a velikosti tlačítek, zaoblení rohů, rozmístění prvků a animace. JavaScript pak zajišťuje veškerou interaktivitu a herní logiku, včetně pohybu vojáků po herní ploše a změn barev buněk.

Backend je postaven na webovém frameworku Django napsaném v jazyce Python. Pro multiplayerový režim jsou využity Django Channels a server Daphne, které umožňují okamžitou obousměrnou komunikaci mezi hráči prostřednictvím protokolu WebSocket. Pro ukládání stavu hry mezi hráči je použit Redis jako rychlá databáze uchovávající data přímo v operační paměti serveru, čímž zajišťuje výrazně rychlejší přístup v porovnání s klasickými databázemi ukládajícími data na pevný disk.

Výběr technologií byl proveden s ohledem na jejich spolehlivost, dostupnost dokumentace a širokou podporu vývojářské komunity. Použité nástroje zároveň umožňují další rozšiřování projektu bez nutnosti zásadních změn architektury. Díky tomu lze v budoucnu implementovat nové funkce s minimálními úpravami stávajícího kódu.

2.2 Struktura souborů

Projekt se opírá o několik hlavních souborů, přičemž každý z nich plní jasně vymezenou roli. Toto rozdělení usnadňuje orientaci v kódu a opravu případných chyb.

Frontend tvoří čtyři hlavní soubory. Soubor `index.html` zobrazuje první stránku, kterou hráč uvidí po otevření hry. Obsahuje hlavní menu se třemi úrovněmi, přihlašovacím formulářem, registrací, tlačítkem pro zobrazení úspěchů a tlačítkem pro multiplayer. Soubor `game.html` představuje herní stránku, na níž se odehrává samotná hra; obsahuje herní plátno, na němž se vykreslují buňky a vojáci, a dále obrazovky pro zobrazení výhry nebo prohry. Soubor `main.js` je z celého frontendu největší a nejdůležitější, neboť obsahuje veškerou herní logiku: od pohybu vojáků po herní ploše, přes dobývání buněk a chování nepřátelské umělé inteligence, až po rozpoznání výhry a prohry. Tento soubor rovněž definuje všechny herní úrovně včetně rozmístění buněk a jejich výchozího počtu vojáků. Soubor `style.css` definuje celkový vzhled hry a obsahuje definice barev tlačítek, velikosti písma, animace a rozmístění všech prvků na stránce.

Backend tvoří rovněž čtyři hlavní soubory. Soubor `views.py` obsahuje serverové funkce, které se spustí při akci hráče, například při přihlášení nebo vytvoření nové multiplayerové hry. Všechny funkce jsou implementovány jako asynchronní pomocí klíčových slov `async` a `await`, takže server může obsluhovat více hráčů současně. Soubor `models.py` definuje strukturu databáze; každý model odpovídá jedné tabulce, jako jsou tabulka hráčů, tabulka odehraných her nebo tabulka herních akcí. Soubor `urls.py` přiřazuje URL adresy konkrétním funkcím: například adresa `/login/` spustí funkci pro přihlášení a adresa `/register/` funkci pro registraci. Soubor `settings.py` obsahuje základní konfiguraci celé aplikace, jako je způsob připojení k databázi, bezpečnostní klíče nebo definice nainstalovaných rozšíření.

Důsledné rozdělení projektu do samostatných souborů výrazně zvyšuje jeho přehlednost. Každý vývojář se může snadněji orientovat v konkrétní části aplikace a rychleji lokalizovat případné chyby. Taková organizace zdrojového kódu je běžným standardem profesionálního vývoje softwaru.

2.3 Frontend: popis herní logiky

Frontend je část aplikace, která běží přímo v prohlížeči hráče a zajišťuje vše, co hráč vidí a s čím interaguje. Veškerá herní logika je implementována v souboru `main.js`. Hra se vykresluje prostřednictvím rozhraní HTML5 Canvas a celá herní smyčka běží plynule díky funkci `requestAnimationFrame`, která zajišťuje optimální frekvenci překreslování pro daný prohlížeč.

Frontend musí reagovat na akce hráče bez znatelného zpoždění. Plynulé vykreslování a rychlé zpracování vstupů mají zásadní vliv na výsledný pocit ze hry. Optimalizace výkonu proto patřila mezi důležité části vývoje.

Buňky

Každá buňka je v kódu reprezentována jako objekt obsahující informace o své pozici na plátně, barvě, aktuálním a maximálním počtu vojáků a o svém vlastníkovi. Vlastník buňky může nabývat tří stavů: hráč, počítač nebo neutrální entita. Na herním plátně se buňka zobrazuje jako kruh s texturou a číslem vojáků uprostřed.

Každá buňka představuje samostatný herní objekt se svými vlastnostmi a chováním. Objektový přístup usnadňuje správu herního stavu a umožňuje jednodušší implementaci nových funkcí. Současně zvyšuje přehlednost celého programu.

Vojáci

Voják se pohybuje vždy od jedné buňky k druhé. Po vytvoření linky hráčem je voják odeslán z výchozí buňky směrem k cílové. Vojáci jsou odesíláni automaticky každých 500 milisekund a pohybují se rychlostí čtyři pixely za snímek. Při dorážení cílové buňky nastávají dvě situace: pokud má voják stejnou barvu jako buňka, buňku posílí; pokud má jinou barvu, buňka ztratí jednoho vojáka za každého přichozího vojáka. Při poklesu počtu vojáků na nulu buňka změní barvu a přejde pod kontrolu útočníka s výchozím stavem pěti vojáků.

Pohyb vojáků je založen na matematických výpočtech směru a vzdálenosti mezi dvěma body. Díky tomu je jejich trajektorie plynulá a nezávislá na velikosti obrazovky. Tento princip zajišťuje konzistentní chování ve všech herních situacích.

Linky

Linka je trvalé spojení mezi dvěma buňkami, po němž jsou vysíláni vojáci. Hráč linku vytvoří kliknutím na vlastní buňku a následným kliknutím na cílovou buňku. Na plátně je linka zobrazena jako čára příslušné barvy. Kliknutím na existující linku ji hráč zruší. Pokud je dobytá buňka, z níž linka vychází, nebo buňka, na níž linka míří, jsou všechny příslušné linky automaticky zrušeny. Hráč může mít aktivních více linek současně, což mu umožňuje útočit na více buněk najednou nebo posilovat obranu vlastních buněk.

Systém linek představuje jednu z nejdůležitějších mechanik celé hry. Umožňuje automatizovat přesuny jednotek a současně zachovat kontrolu nad jejich směřováním. Hráč tak může plánovat více akcí současně a efektivněji reagovat na vývoj situace.

Regenerace buněk

Buňka samostatně regeneruje jednoho vojáka každých 800 milisekund za předpokladu, že v daném intervalu nevysílá ani nepřijímá žádného vojáka. Hráč může tento mechanismus takticky využívat pro obnovu oslabených buněk. Regenerace platí pro všechny typy buněk bez ohledu na barvu.

Mechanismus regenerace zabraňuje příliš rychlému vyčerpání zdrojů. Hráč je motivován nejen k útoku, ale také k promyšlené obraně a správnému hospodaření s jednotkami. Tím vzniká vyváženější herní prostředí.

Chování počítačového nepřítele

Počítačový nepřítel vysílá každých 800 milisekund vojáka na náhodně vybranou cílovou buňku. Podmínkou odeslání vojáka je, aby měla fialová buňka alespoň pět vojáků. Výběr cíle je čistě náhodný a nepřítel může útočit jak na modré neutrální, tak na zelené hráčovy buňky.

I přes relativně jednoduchou implementaci dokáže nepřítel vytvářet nepředvídatelné situace. Náhodný výběr cílů přispívá k variabilitě jednotlivých herních partií. Hráč se proto nemůže spoléhat na stále stejný postup.

Animace výhry

Po splnění úrovně se spustí animace zlatých konfet. Každý snímek vytvoří s třicetiprocentní pravděpodobností nový konfet na náhodném místě horního okraje plátna. Každý konfet má náhodně přiřazenou rychlost a velikost. Animace automaticky nezavírá výherní obrazovku a slouží jako vizuální odměna hráči za úspěšné dokončení úrovně.

Vizuální odměna po dokončení úrovně zvyšuje pocit úspěchu a motivuje hráče k pokračování ve hře. Animace zároveň slouží jako jasný indikátor splnění herního cíle. Podobné prvky jsou běžně využívány v moderním herním designu.

2.4 Backend: popis serverové části

Backend je aplikace běžící na serveru v pozadí. Hráč ji přímo nevidí, ale komunikuje s ní například při přihlášení nebo při hraní v multiplayerovém režimu.

Serverová část aplikace zajišťuje všechny operace, které není vhodné provádět přímo v prohlížeči. Oddělení klientské a serverové logiky zvyšuje bezpečnost i celkovou stabilitu systému. Tento přístup odpovídá současným standardům webového vývoje.

Registrace a přihlášení

Při registraci odešle prohlížeč vyplněné údaje na server. Server nejprve zkontroluje délku uživatelského jména, poté ověří bezpečnost hesla dle Django pravidel. Po úspěšné validaci je heslo zahashováno a spolu s ostatními údaji uloženo do databáze. Heslo se nikdy neukládá v čitelné podobě. Přihlášení funguje obdobně: hráč zadá uživatelské jméno nebo e-mailovou adresu a heslo, server zadané heslo zahashuje stejným algoritmem a porovná s uloženým hashem. Po úspěšném přihlášení může hráč zahájit hru. Správná správa uživatelských účtů je důležitá nejen z hlediska pohodlí hráče, ale také z hlediska bezpečnosti. Ověřování údajů pomáhá předcházet neoprávněnému přístupu k účtům. Současně umožňuje ukládání herního postupu mezi jednotlivými relacemi.

Databáze

Databáze je místo, kde jsou trvale uloženy všechny informace o hráčích a jejich hrách. V projektu je použita databáze SQLite, která ukládá veškerá data do jediného souboru na serveru. Databáze obsahuje oddělené tabulky pro různé typy dat: tabulku hráčů, tabulku herních relací s uloženým stavem hry ve formátu JSON a tabulku herních akcí zaznamenávající jednotlivé akce hráčů.

Databázový systém představuje centrální úložiště všech důležitých informací. Správně navržená databázová struktura usnadňuje ukládání i následné vyhledávání dat. Její návrh proto patří mezi klíčové části každé webové aplikace.

Zabezpečení

Aplikace je zabezpečena na několika úrovních. Každý formulář je chráněn tokenem CSRF, což je speciální kód zabraňující útokům, při nichž by cizí stránka mohla odesílat formuláře jménem přihlášeného hráče bez jeho vědomí. Hesla jsou ukládána výhradně v zahashované podobě dle pravidel frameworku Django. Všechny citlivé operace, jako je změna hesla nebo uživatelských údajů, jsou navíc chráněny kontrolou oprávnění přihlášeného uživatele.

Bezpečnost webových aplikací je v současné době jedním z nejdůležitějších aspektů vývoje. Ochrana uživatelských dat musí být zajištěna na několika úrovních současně. Použitá bezpečnostní opatření významně snižují riziko zneužití systému.

Asynchronní serverové funkce

Všechny serverové funkce jsou implementovány jako asynchronní, takže server může zpracovávat více požadavků současně bez čekání na dokončení předchozího. Tato vlastnost je klíčová zejména pro multiplayerový režim, kde více hráčů komunikuje se serverem ve stejný čas. Každá funkce přijme požadavek, zpracuje ho, provede potřebné operace s databází a odešle jednoduchou textovou odpověď, například "success" při úspěchu nebo "fail" při chybě.

Asynchronní programování umožňuje efektivnější využití serverových prostředků. Server nemusí čekat na dokončení jedné operace a může mezitím zpracovávat další požadavky. To je důležité zejména při větším počtu současně připojených hráčů.

2.5 Komunikace frontendu a backendu

Frontend a backend jsou dvě oddělené části, které musejí vzájemně komunikovat. Bez této komunikace by nefungovaly základní funkce, jako je přihlášení ani multiplayerový režim.

HTTP komunikace

HTTP je základní způsob komunikace na internetu. Funguje na principu dotaz a odpověď: prohlížeč odešle požadavek na server a server odpoví. Tento způsob se využívá pro standardní operace, jako je registrace, přihlášení, odhlášení nebo načtení stránky. Nevýhodou HTTP je skutečnost, že vždy musí prohlížeč iniciovat komunikaci jako první; server nemůže sám od sebe zaslat zprávu prohlížeči. Pro běžné operace to nevadí, avšak pro hru v reálném čase vyžadující okamžitou obousměrnou komunikaci je toto omezení nedostačující. Každý HTTP požadavek má vlastní URL adresu definovanou v souboru `urls.py` a přiřazenou konkrétní funkci v souboru [views.py](#). Komunikace pomocí HTTP tvoří základ fungování většiny webových aplikací. Přestože je tento model velmi spolehlivý, není vhodný pro přenos dat v reálném čase. Proto je v projektu doplněn modernějšími technologiemi.

WebSocket komunikace

WebSocket je modernější komunikační protokol, který odstraňuje zmíněné omezení HTTP. Po navázání spojení mohou obě strany, prohlížeč i server, odesílat zprávy kdykoli bez čekání na druhou stranu. Funguje to analogicky jako telefonický hovor: obě strany mohou hovořit kdykoli chtějí. Tato vlastnost je zásadní pro multiplayerový režim, kde hráč musí okamžitě vidět akce soupeře. WebSocket spojení zůstává otevřené po celou dobu hry a uzavře se až při opuštění herní stránky nebo zavření prohlížeče. WebSocket výrazně zkracuje dobu potřebnou pro přenos informací mezi klientem a serverem. Díky trvale otevřenému spojení mohou být změny přenášeny prakticky okamžitě. To je zásadní zejména u multiplayerových her.

Průběh multiplayerové komunikace

Celý průběh komunikace během multiplayerové hry probíhá takto. Nejprve hráč vytvoří novou hru prostřednictvím klasického HTTP požadavku a server mu vrátí identifikační číslo herní relace. Poté se prohlížeč hráče připojí na speciální WebSocket adresu obsahující toto číslo. Druhý hráč provede totéž a připojí se na stejnou adresu. Od tohoto okamžiku jsou všechny herní akce, jako je vytváření linek nebo dobývání buněk, odesílány přes WebSocket na server, který je okamžitě přeposílá druhému hráči. Díky tomu oba hráči vidí shodnou herní situaci ve stejný čas. Django Channels zajišťuje správu všech WebSocket spojení, server Daphne umožňuje jejich souběžné zpracování a Redis slouží jako rychlý prostředník pro přeposílání zpráv mezi hráči. Synchronizace herního stavu mezi více hráči patří mezi technicky nejnáročnější části projektu. Je nutné zajistit, aby všichni účastníci viděli stejný průběh hry ve stejném okamžiku. K tomu slouží kombinace WebSocketů, Django Channels a databázových mechanismů

3 Grafika a design

3.1 Grafický koncept buněk

Základní herní logika rozděluje buňky do tří typů na základě barev: zelené ovládané hráčem, fialové ovládané počítačem a modré neutrální. Aby hra působila profesionálnějším a organičtějším dojmem, byly tyto základní barevné kruhy nahrazeny detailními dvourozměrnými texturami. Každý typ buňky dostal vlastní vizuální charakter, od klidných a hladkých tvarů pro hráče a neutrální entity až po agresivnější špičaté okraje pro buňky nepřátelské.

3.2 Technické zpracování textur

Při tvorbě grafických podkladů bylo důsledně dodržováno pravidlo mocnin dvou (tzv. Power of Two). Herní enginy a webové prohlížeče dokážou textury v těchto rozměrech zpracovávat výrazně efektivněji. Všechny hlavní buňky byly nakresleny ve čtvercovém rozlišení 256×256 pixelů a následně jsou přímo v herním kódu zmenšovány na požadovaný herní poloměr. Tento přístup, tedy kreslení ve vyšším rozlišení a následné zmenšení, zaručuje ostrost grafiky i na moderních monitorech s vysokou hustotou pixelů. Pro zachování průhledného pozadí a poloprůhledných zářivých okrajů byl použit formát PNG s alfa kanálem.

Při kreslení textur byl záměrně ponechán volný prostor (angl. padding) mezi samotnou buňkou a okrajem plátna. Tento prostor slouží jako rezerva pro animace, například pulzování, aby nedocházelo k nežádoucímu ořezávání okrajů grafiky při jejím zvětšení během animační smyčky.

3.3 Animace pomocí spritesheetů

Aby buňky nepůsobily staticky, byly do hry implementovány animace. Rozhraní HTML5 Canvas nativně nepodporuje přehrávání animovaných formátů, jako je GIF, bez ztráty kvality a průhlednosti. Z tohoto důvodu byla zvolena průmyslová metoda takzvaných spritesheetů. Všechny fáze pohybu dané animace byly exportovány a seřazeny vedle sebe do jednoho dlouhého PNG obrázku. Herní smyčka prostřednictvím funkce `requestAnimationFrame` posouvá výřez, kterým se na tento pás nahlíží, čímž vzniká iluze plynulého pohybu. Tento způsob je paměťově nenáročný a zachovává ostrost poloprůhledných stínů.

3.4 Grafika vojáků

Vojáci představují jednotky pohybující se plynule od jedné buňky k druhé. V pokročilých fázích hry jich může být na plátně současně desítky až stovky. Z důvodu optimalizace výkonu bylo pro vojáky zvoleno menší rozlišení plátna 32×32 pixelů. Grafika vojáků je vycentrována přesně na střed, aby nedocházelo k vizuálním odchylkám při jejich pohybu po vypočítaných matematických trajektoriích. Vizuální styl střel je zjednodušený, avšak zachovává jasnou barevnou identifikaci příslušnosti k hráči nebo nepříteli.

3.5 Herní prostředí a pozadí

Pro umocnění mikroskopického dojmu tzv. efektu Petriho misky bylo navrženo organické pozadí. Místo jednoho velkého statického obrázku, který by nadměrně zatěžoval operační paměť, byla vytvořena opakující se (bezešvá) textura o velikosti 512×512 pixelů. Při její tvorbě byla využita metoda posunu (angl. *offset*), která zajišťuje plynulé navazování okrajů obrázku ve všech směrech. Prostřednictvím metody `createPattern` na herním plátně se tato dlaždice donekonečna opakuje a vyplní celou hrací plochu, aniž by hráč zaznamenal jakékoli přechody.

4 Využití umělé inteligence při vývoji

Při vývoji projektu byly využity nástroje umělé inteligence jako pomocný prostředek pro řešení problémů, s nimiž si vývojáři nevěděli rady. Konečná implementace a ověření správnosti však vždy probíhaly ze strany autorů projektu. Hlavními nástroji byl jazykový model Claude od společnosti Anthropic a ChatGPT od společnosti OpenAI. Tyto nástroje byly využívány cíleně pro konkrétní technické problémy, nikoli jako prostředek pro generování celých bloků kódu.

4.1 Detekce bodu u úsečky

Prvním řešeným problémem byla matematická funkce pro detekci bodu v blízkosti úsečky. Tato funkce využívá přesný výpočet vzdálenosti bodu od úsečky pomocí vektorové projekce, skalárního součinu a parametrizace přímky.

Funkce pracuje následovně. Nejprve se vypočítají dva vektory: vektor v reprezentuje směr úsečky od bodu A do bodu B a vektor w reprezentuje směr od bodu A k testovanému bodu. Pomocí skalárního součinu $c1 = v \cdot w$ a $c2 = v \cdot v$ se získá parametr $t = c1/c2$, který udává, kde se na úsečce nachází nejbližší bod. Hodnota $t = 0$ odpovídá bodu A a $t = 1$ odpovídá bodu B . Metodou `Math.max(0, Math.min(1, t))` je parametr oříznut na interval $\langle 0, 1 \rangle$, aby nejbližší bod nevypadl mimo úsečku. Nakonec `Math.hypot` změří euklidovskou vzdálenost mezi testovaným bodem a projekcí a porovná ji s definovanou mezní hodnotou.

```
function isPointNearLine(x, y, a, b, threshold = 6) { const vx = b.x - a.x; {
const vx = b.x - a.x;
const vy = b.y - a.y;
const wx = x - a.x;
const wy = y - a.y;
const c1 = vx * wx + vy * wy;
const c2 = vx * vx + vy * vy;
let t = 0;
if (c2 !== 0) t = c1 / c2;
t = Math.max(0, Math.min(1, t));
const px = a.x + t * vx;
const py = a.y + t * vy;
const dist = Math.hypot(x - px, y - py);
return dist <= threshold;
}
```

4.2 Bezpečné načítání obrázků pomocí Promise

Druhým příkladem využití umělé inteligence byla funkce pro bezpečné načítání obrázků pomocí rozhraní Promise. Funkce ošetřuje tři různé stavy: okamžitě načtený obrázek, úspěšné načtení po prodlevě a chybu při načítání.

Funkce obalí nativní DOM události do vzoru Promise, tzv. promisifikace. Objekt Image disponuje vlastností `complete`, která nabývá hodnoty `true` v případě zpracování obrázku z mezipaměti; v takovém případě se `resolve()` zavolá ihned. V ostatních případech jsou zaregistrovány dva posluchače událostí: `load` pro úspěšné načtení a `error` pro selhání. Oba zavolají `resolve()`, protože funkce vyžaduje pouze dokončení operace, nikoli rozlišení jejího výsledku. Funkce se volá v rámci `Promise.all()`, který čeká na dokončení všech operací souběžně a teprve poté spustí funkci `draw()`.

```
function nactiObrazek(img) {
  return new Promise(function(resolve) {
    if (img.complete) {
      resolve();
      return;
    }
    img.addEventListener('load', function() {
      resolve();
    });
    img.addEventListener('error', function() {
      console.warn("Obrázek se nepodarilo načíst: " + img.src);
      resolve();
    });
  });
}
```

4.3 Pravidelné odesílání vojáků pomocí setInterval

Třetím příkladem je systém pravidelného odesílání vojáků mezi buňkami prostřednictvím funkce `setInterval`. Kód průběžně ukládá reference na intervaly pro jejich pozdější rušení a kontroluje duplicity pomocí metody `Array.some()`.

Každá linka je implementována jako samostatný interval s frekvencí 500 ms. Před vytvořením nové linky se pomocí `Array.some()` ověřuje existence duplicitní linky porovnáváním referencí objektů

from a to; neporovnávají se jejich hodnoty, nýbrž přímo reference. Reference na intervalId je uložena do objektu v poli activeLinks, aby bylo možné interval kdykoli zrušit voláním clearInterval. Bez uložení identifikátoru by interval pokračoval v běhu v paměti i po odebrání linky ze seznamu. Podmínka if (!gamePaused) uvnitř zpětného volání zajišťuje, že interval sice tiká, ale vojáci jsou odesílání pouze tehdy, když hra běží.

```
function startAutoSend(from, to) {  
  if (activeLinks.some(function(link) {  
    return link.from === from && link.to === to;  
  })) return;  
  
  const interval = setInterval(function() {  
    if (!gamePaused) {  
      endSoldiers (from, to);  
    }  
  }, 500);  
  
  activateLinks.push({ from: from, to: to, interval: interval });  
}
```

5 Srovnání s podobnými hrami

Herní žánr, do nějž hra Brain Cell Wars spadá, se nazývá RTS neboli strategie v reálném čase (z angl. Real-Time Strategy). Na trhu existuje řada her s podobným konceptem. Hra Spore ve své počáteční buněčné fázi sdílí myšlenku ovládání jednobuněčného organismu a rozšiřování jeho území. V mobilním světě existují hry jako Galcon nebo Tentacle Wars, které fungují na velmi podobném principu vysílání jednotek mezi planetami nebo buňkami. Všechny tyto hry sdílejí základní myšlenku: ovládání území prostřednictvím vysílání jednotek. Brain Cell Wars tuto myšlenku rozvíjí o organické vizuální zpracování a mechanismus trvalých linek, který snižuje nutnost opakovaného manuálního zadávání příkazů. Analýza existujících her pomohla identifikovat osvědčené herní principy a současně odhalit oblasti, ve kterých se lze od konkurence odlišit. Inspirace jinými projekty je běžnou součástí herního vývoje. Důležité je však vytvořit vlastní originální kombinaci mechanik a vizuálního stylu.

6 Závěr

6.1 Co jsme se naučili

Práce na projektu Brain Cell Wars přinesla cenné zkušenosti napříč celým spektrem webového vývoje. Na straně frontendu jsme se naučili pracovat s rozhraním HTML5 Canvas, vykreslovat herní objekty pomocí JavaScriptu, implementovat herní smyčku pomocí requestAnimationFrame a navrhovat systém ukládání dat v prohlížeči. Pochopili jsme, jak funguje komunikace mezi více JavaScriptovými soubory a jak správně řídit pořadí jejich načítání.

Na straně backendu jsme získali první praktické zkušenosti s frameworkem Django a programovacím jazykem Python. Naučili jsme se navrhovat databázové modely, psát asynchronní serverové funkce a implementovat zabezpečení webové aplikace. Největší výzvou a zároveň největším přínosem bylo propojení frontendu s backendem prostřednictvím protokolu WebSocket pro multiplayerový režim, který nám otevřel pohled na fungování moderních webových aplikací v reálném čase.

6.2 Možnosti dalšího rozvoje

Projekt má několik oblastí vhodných k dalšímu rozvoji. Z hlediska hratelnosti by bylo zajímavé implementovat sofistikovanější umělou inteligenci počítačového nepřítele, která by útočila strategicky namísto náhodně. Dalším vylepšením by mohlo být přidání více typů buněk s různými schopnostmi nebo speciálních útoků přidávajících hře taktickou hloubku.

Z technického hlediska by bylo vhodné přesunout ukládání herního postupu z localStorage na server, což by hráčům umožnilo přistupovat ke svému postupu z více zařízení. Grafická stránka hry by mohla být dále rozvinuta dokončením implementace spritesheetových animací buněk, jejíž základy byly v projektu položeny, avšak plně realizovány nebyly.

6.3 Zhodnocení projektu

Projekt Brain Cell Wars splnil všechny cíle stanovené na jeho začátku. Podařilo se vytvořit funkční prohlížečovou hru s třemi herními úrovněmi, systémem přihlašování a ukládání postupu a připravenou infrastrukturou pro multiplayerový režim. Projekt nás výrazně posunul v programátorských dovednostech a ukázal nám, jak vypadá vývoj skutečné webové aplikace od návrhu až po implementaci. Naučili jsme se pracovat v týmu, rozdělovat práci a řešit problémy přicházející v průběhu vývoje.

Zdroje

- Django SOFTWARE FOUNDATION. Cross Site Request Forgery protection. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/csrf/> [cit. 2025-12-12].
- Django SOFTWARE FOUNDATION. Built-in template tags and filters. In: Django documentation[online].Dostupné z:<https://docs.djangoproject.com/en/6.0/ref/templates/builtins/> [cit. 2025-12-20].
- Django SOFTWARE FOUNDATION. Sending email. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/email/> [cit. 2026-01-05].
- Django SOFTWARE FOUNDATION. Asynchronous support. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/async/> [cit. 2026-01-18].
- Django SOFTWARE FOUNDATION. URL dispatcher. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/urls/> [cit. 2026-02-02].
- Django SOFTWARE FOUNDATION. Views. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/views/> [cit. 2026-02-10].
- Django SOFTWARE FOUNDATION. View decorators. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/decorators/> [cit. 2026-02-18].
- Django SOFTWARE FOUNDATION. Database queries. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/queries/> [cit. 2026-02-25].
- Django SOFTWARE FOUNDATION. Aggregation. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/aggregation/> [cit. 2026-03-03].
- Django SOFTWARE FOUNDATION. Transactions. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/transactions/> [cit. 2026-03-10].
- Django SOFTWARE FOUNDATION. Optimization. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/optimization/> [cit. 2026-03-18].
- Django SOFTWARE FOUNDATION. Fixtures. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/fixtures/> [cit. 2026-03-25].
- Django SOFTWARE FOUNDATION. Request and response objects. In: Django documentation[online].Dostupné z:<https://docs.djangoproject.com/en/6.0/ref/request-response/> [cit. 2026-04-01].
- Django SOFTWARE FOUNDATION. Model field reference. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/models/fields/> [cit. 2026-04-08].
- Django SOFTWARE FOUNDATION. Settings. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/settings/> [cit. 2026-04-15].
- Django SOFTWARE FOUNDATION. Authentication system. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/contrib/auth/> [cit. 2026-04-20].

- Django SOFTWARE FOUNDATION. Using the authentication system. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/auth/default/> [cit. 2026-04-25].
- Django SOFTWARE FOUNDATION. Customizing authentication. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/auth/customizing/> [cit. 2026-04-30].
- Django Channels CHANNELS PROJECT. Tutorial. In: Channels documentation [online]. Dostupné z: <https://channels.readthedocs.io/en/latest/tutorial/index.html> [cit. 2026-05-01].
- Django Channels CHANNELS PROJECT. Database access. In: Channels documentation [online]. Dostupné z: <https://channels.readthedocs.io/en/latest/topics/databases.html> [cit. 2026-05-03].
- Mozilla MOZILLA. *Canvas API*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API [cit. 2025-12-15].
- Mozilla MOZILLA. *Window: requestAnimationFrame() method*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame> [cit. 2025-12-22].
- Mozilla MOZILLA. *Window: localStorage property*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> [cit. 2026-01-08].
- Mozilla MOZILLA. *WebSocket API*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API [cit. 2026-01-20].
- Django Channels CHANNELS PROJECT. *Introduction*. In: Channels documentation [online]. Dostupné z: <https://channels.readthedocs.io/en/stable/introduction.html> [cit. 2026-01-28].
- Mozilla MOZILLA. *Promise.all()*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all [cit. 2026-02-05].
- Mozilla MOZILLA. *setInterval() global function*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/setInterval> [cit. 2026-02-12].
- Django DJANGO SOFTWARE FOUNDATION. *Django documentation*. [online]. Dostupné z: <https://docs.djangoproject.com/en/stable/> [cit. 2026-02-20].
- Mozilla MOZILLA. *CanvasRenderingContext2D: globalCompositeOperation property*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/globalCompositeOperation> [cit. 2026-03-01].
- **ChatGPT — konverzační AI** OpenAI. *ChatGPT*. Dostupné z: <https://chat.openai.com/> [cit. 2026-03-18]
- **Claude — konverzační AI** Anthropic. *Claude AI*. Dostupné z: <https://claude.ai/> [cit. 2025-12-09]